ECE Paris – OMNES Education
Big Data Frameworks · ING4 Data & IA

Compte rendu – Lab Spark DataFrame

Bronze → Silver avec Spark, Parquet et Iceberg

Session PySpark connectée à S3, couche silver construite avec l'API DataFrame et avec Spark SQL, écriture en Parquet et en tables Iceberg, comparaison avec le lab dbt et exercices

Auteure	Dorline TIAKO OKASSIE
Groupe	ING4 – Groupe 01 – Data & Intelligence Artificielle

Environnement	Service Onyxia `vscode-pyspark` (4 CPU / 4000 Mi), Spark 4.1.1, Scala 2.13, Iceberg 1.11.0, pandas 3.0.5, pyarrow 25.0.1
Bucket	`s3://user-d-tiakookassie-ece`
Dépôt Git	`github.com/Dorline0121/ece-2026-bigdata`
Date	5 novembre 2026

Sommaire

1. Objectifs du lab
2. Difficultés rencontrées et adaptations
3. Environnement et session Spark
4. Lecture de la couche bronze
5. Couche silver avec l'API DataFrame
6. La même transformation en Spark SQL
7. Écriture de la couche silver en Parquet
8. Écriture de la couche silver en tables Iceberg
9. Comparaison avec le lab dbt
10. Exercices
11. Synthèse et conclusion

1. Objectifs du lab

- Connecter une `SparkSession` PySpark au stockage objet S3 d'Onyxia.
- Lire les fichiers CSV de la couche bronze (`users.csv`, `orders.csv`) directement sur S3.
- Construire **la même couche silver que dans le lab dbt**, avec l'API DataFrame puis avec Spark SQL.
- Écrire cette couche silver sur S3, en fichiers Parquet puis en tables Iceberg, et examiner la structure d'une table Iceberg.
- Comparer les deux approches : dbt avec DuckDB, et Spark.

2. Difficultés rencontrées et adaptations

① Session Spark déjà créée par le shell

Constat. Le shell `pyspark` crée lui-même une session `spark` au démarrage. Or les réglages S3A (`spark.hadoop.*`) et les extensions Iceberg (`spark.sql.extensions`) ne s'appliquent qu'à la **création** d'une session : un `getOrCreate()` sur la session existante les aurait ignorés.

Adaptation. Arrêt de la session automatique avec `spark.stop()`, puis création d'une nouvelle session avec toute la configuration. Sa prise en compte a été vérifiée (figure 6).

② Collage de code dans le shell Python

Constat. Deux blocs collés à la suite ont fusionné deux `print` sur une même ligne, et une fonction `def` sans ligne vide finale a provoqué une `SyntaxError`. Des commandes bash collées par erreur après `>>>` ont aussi été refusées.

Adaptation. Code découpé en petits blocs, fonctions écrites en une ligne (`lambda`), et petite fonction `sh()` fondée sur `subprocess` pour lancer les commandes `aws` sans quitter PySpark. Un second terminal bash a servi pour Git.

③ Écarts avec le sujet

- **Volume** : le sujet annonce 5000 commandes (5003 avec doublons) ; le jeu de données utilisé en compte 2829 (2832 avec doublons).
- **Chemins dbt** : les fichiers exportés au lab dbt s'appellent `silver/dbt/orders.parquet` et `users.parquet`, et non `stg_*.parquet` : les requêtes de comparaison ont été adaptées.
- **Snapshot Iceberg** : `createOrReplace()` produit un premier snapshot de type `overwrite`, et non `append`.
- `%%timeit` n'existe que dans Jupyter : les mesures ont été faites avec `time.perf_counter()` et une écriture vers le format `noop`, qui force le calcul complet sans rien écrire.
- **Authentification Git** : la connexion GitHub de VS Code ne fonctionnait plus dans le terminal ; le `git push` a été fait avec un *personal access token*.

3. Environnement et session Spark

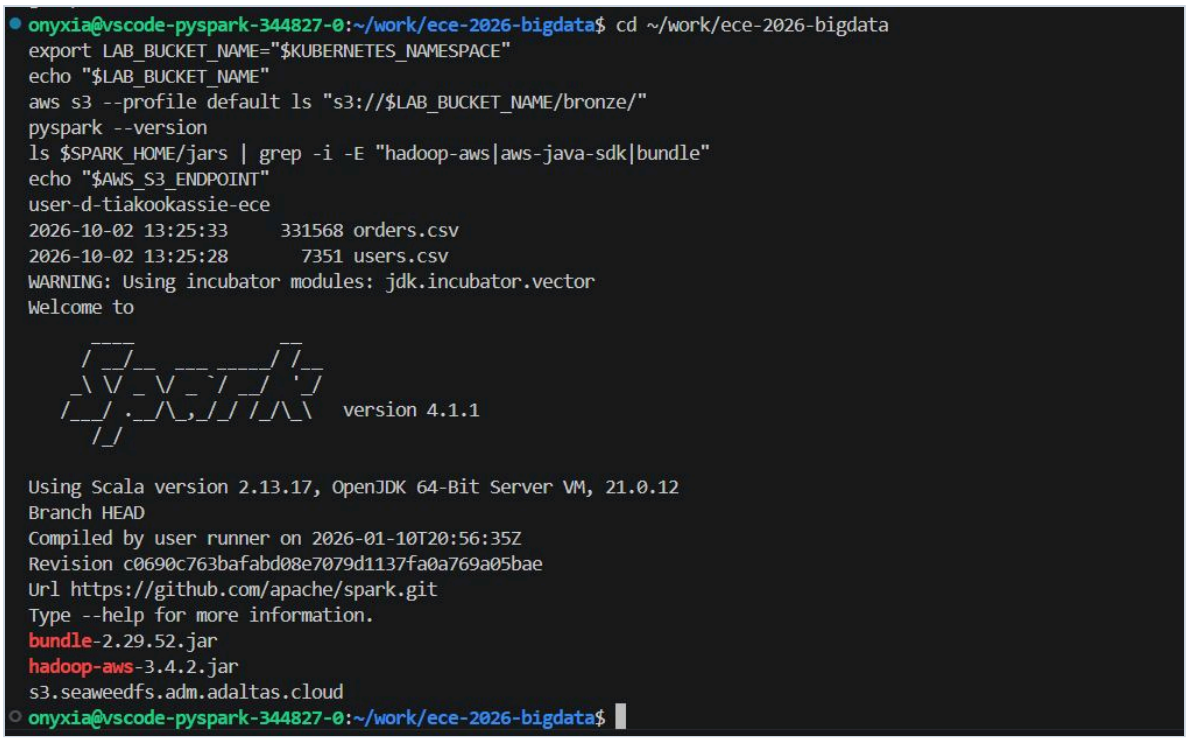


Figure 1 – Bucket et couche bronze présents ; Spark 4.1.1 avec Scala 2.13 ; bibliothèques hadoop-aws 3.4.2 et SDK AWS (bundle 2.29.52) ; endpoint SeaweedFS

Le paquet Iceberg doit correspondre aux versions de Spark et de Scala : `iceberg-spark-runtime-4.1_2.13:1.11.0`, celui du sujet, convient sans adaptation.

```
pyspark --master "local[*]" --packages org.apache.iceberg:iceberg-spark-runtime-4.1_2.13:1.11.0
```

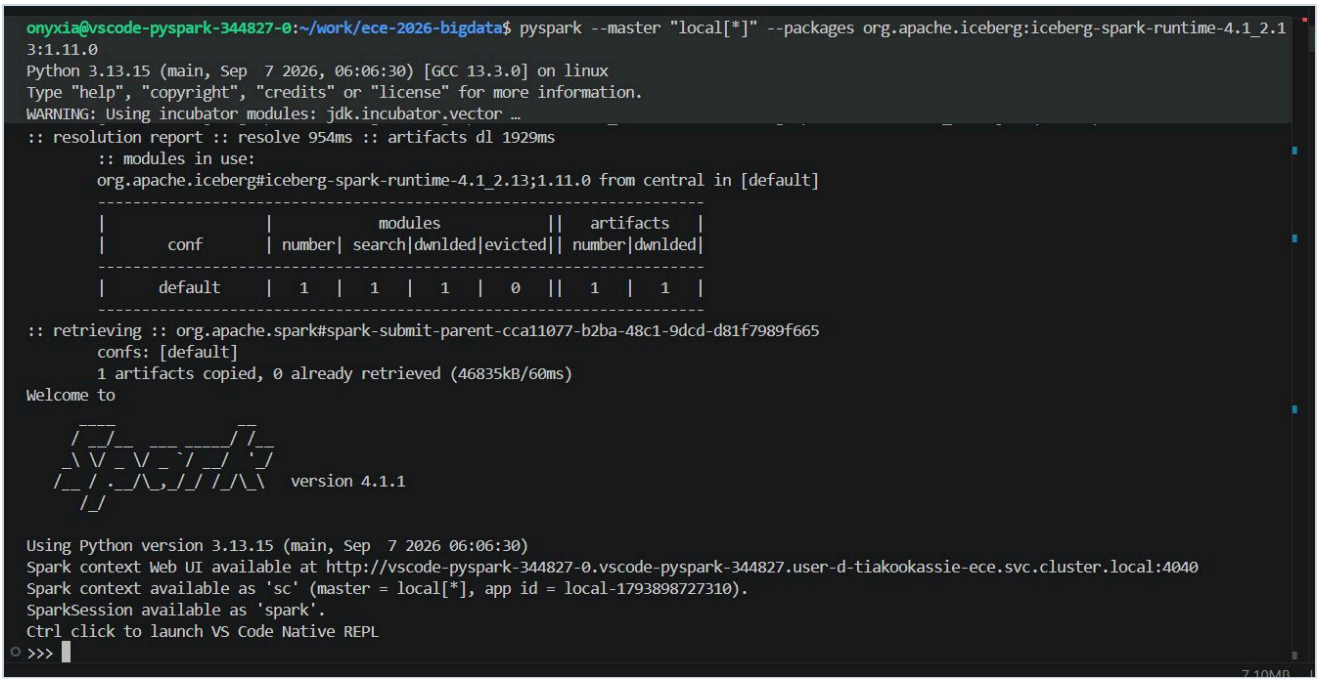


Figure 2 – Lancement du shell PySpark : téléchargement d'Iceberg depuis Maven Central et démarrage de Spark 4.1.1 en mode local[*]

```
spark.stop()   # la session automatique ignorerait la configuration S3A et Iceberg
spark = (SparkSession.builder.appName("lab-silver-pyspark").master("local[*]")
    .config("spark.hadoop.fs.s3a.endpoint", f"https://{os.environ['AWS_S3_ENDPOINT']}")
    .config("spark.hadoop.fs.s3a.access.key", os.environ["AWS_ACCESS_KEY_ID"])
    .config("spark.hadoop.fs.s3a.secret.key", os.environ["AWS_SECRET_ACCESS_KEY"])
    .config("spark.hadoop.fs.s3a.session.token", os.environ.get("AWS_SESSION_TOKEN", ""))
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
            "org.apache.hadoop.fs.s3a.TemporaryAWSCredentialsProvider")
    .config("spark.hadoop.fs.s3a.path.style.access", "true")
    .config("spark.sql.extensions",
            "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
    .config("spark.sql.catalog.lab", "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.lab.type", "hadoop")
    .config("spark.sql.catalog.lab.warehouse", f"s3a://{BUCKET}/warehouse")
    .getOrCreate())
```

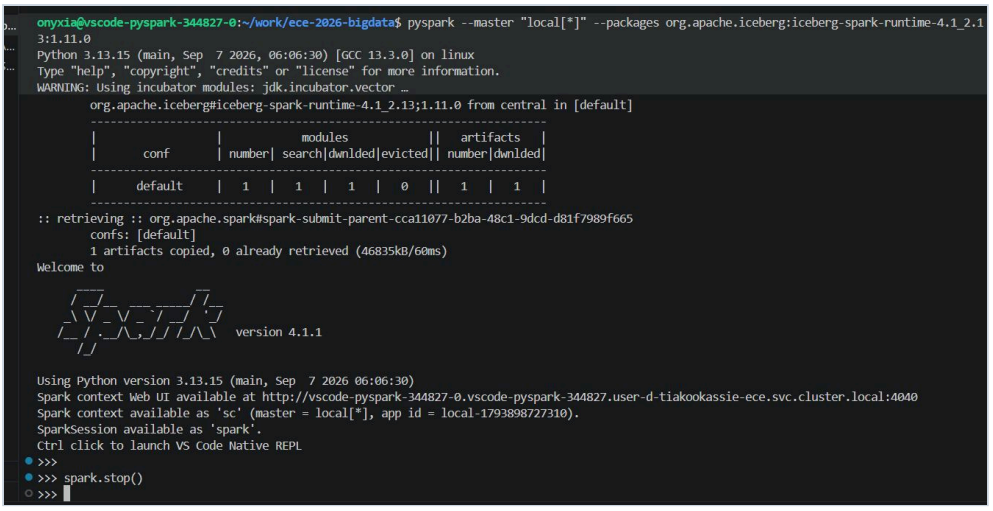


Figure 3 – Arrêt de la session créée automatiquement par le shell (difficulté ①)

```
>>> os.environ["LAB_BUCKET_NAME"] = os.environ["KUBERNETES_NAMESPACE"]
>>> BUCKET = os.environ["LAB_BUCKET_NAME"]
>>> ICEBERG_PACKAGE = "org.apache.iceberg:iceberg-spark-runtime-4.1_2.13:1.11.0"
>>> spark = (
...     SparkSession.builder
...     .appName("lab-silver-pyspark")
...     .master("local[*]")
...     .config("spark.hadoop.fs.s3a.endpoint", f"https://{os.environ['AWS_S3_ENDPOINT']}")
...     .config("spark.hadoop.fs.s3a.access.key", os.environ["AWS_ACCESS_KEY_ID"])
...     .config("spark.hadoop.fs.s3a.secret.key", os.environ["AWS_SECRET_ACCESS_KEY"])
...     .config("spark.hadoop.fs.s3a.session.token", os.environ.get("AWS_SESSION_TOKEN", ""))
...     .config("spark.hadoop.fs.s3a.aws.credentials.provider", "org.apache.hadoop.fs.s3a.TemporaryAWSCredentialsProvider")
...     .config("spark.hadoop.fs.s3a.path.style.access", "true")
...     .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
...     .config("spark.jars.packages", ICEBERG_PACKAGE)
...     .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
...     .config("spark.sql.catalog.lab", "org.apache.iceberg.spark.SparkCatalog")
...     .config("spark.sql.catalog.lab.type", "hadoop")
...     .config("spark.sql.catalog.lab.warehouse", f"s3a://{BUCKET}/warehouse")
...     .getOrCreate()
... )
>>> spark.sparkContext.setLogLevel("ERROR")
>>> print(spark, spark.version, BUCKET)print(spark.conf.get("spark.sql.extensions"))
  File "<stdin>", line 1
    print(spark, spark.version, BUCKET)print(spark.conf.get("spark.sql.extensions"))
                                       ^^^^^
SyntaxError: invalid syntax
>>> print(spark.sparkContext._jsc.hadoopConfiguration().get("fs.s3a.endpoint"))
https://s3.seaweedfs.adm.adaltas.cloud
>>>
```

Figure 4 – Création de la session configurée ; la SyntaxError vient de deux print fusionnés au collage (difficulté ②)

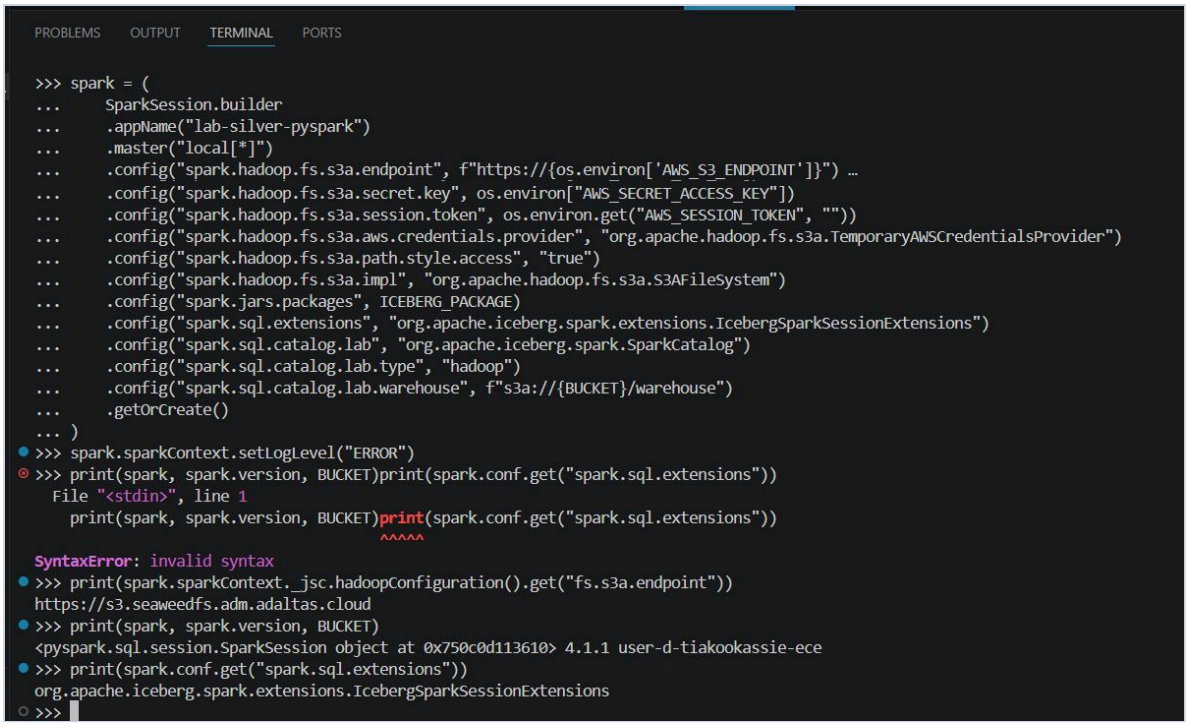


Figure 5 – Vérification : session Spark 4.1.1, extensions Iceberg actives et endpoint S3 configuré

Réponses aux questions

Q1. Les identifiants sont temporaires. Qu'arrive-t-il à un job Spark en cours si le service est redémarré, et pourquoi ?

Le job est **tué**. En mode `local[*]`, le driver et les exécuteurs tournent dans la JVM du conteneur du service : redémarrer le service arrête la session, avec ses données en mémoire et en cache. Au redémarrage, Onyxia délivre de **nouveaux identifiants STS**, mais ceux de l'ancienne session avaient été lus une seule fois, à sa création. Même si le job survivait, il utiliserait des identifiants qui finissent par expirer, et ses requêtes S3 échoueraient en `403` / `ExpiredToken`. Il faut relancer le job dans une nouvelle session. Une écriture interrompue peut laisser des fichiers partiels dans un dossier Parquet, alors qu'Iceberg n'expose jamais une écriture inachevée.

Q2. Pourquoi un SimpleAWSCredentialsProvider échoue-t-il ici, alors qu'il fonctionnait sans session token ?

`SimpleAWSCredentialsProvider` n'envoie que la clé d'accès et la clé secrète. Or les identifiants d'Onyxia sont des identifiants **temporaires STS**, valides seulement **accompagnés du session token** : sans lui, le serveur rejette la signature (`403`, `InvalidAccessKeyId` ou `InvalidToken`). `TemporaryAWSCredentialsProvider` lit en plus `fs.s3a.session.token` et l'ajoute à chaque requête. Avec des clés permanentes, il n'y a pas de token, et le fournisseur simple suffit.

4. Lecture de la couche bronze

```
users_bronze = spark.read.csv(f"s3a://{BUCKET}/bronze/users.csv", header=True,
    inferSchema=True, samplingRatio=0.01, multiLine=True)
```

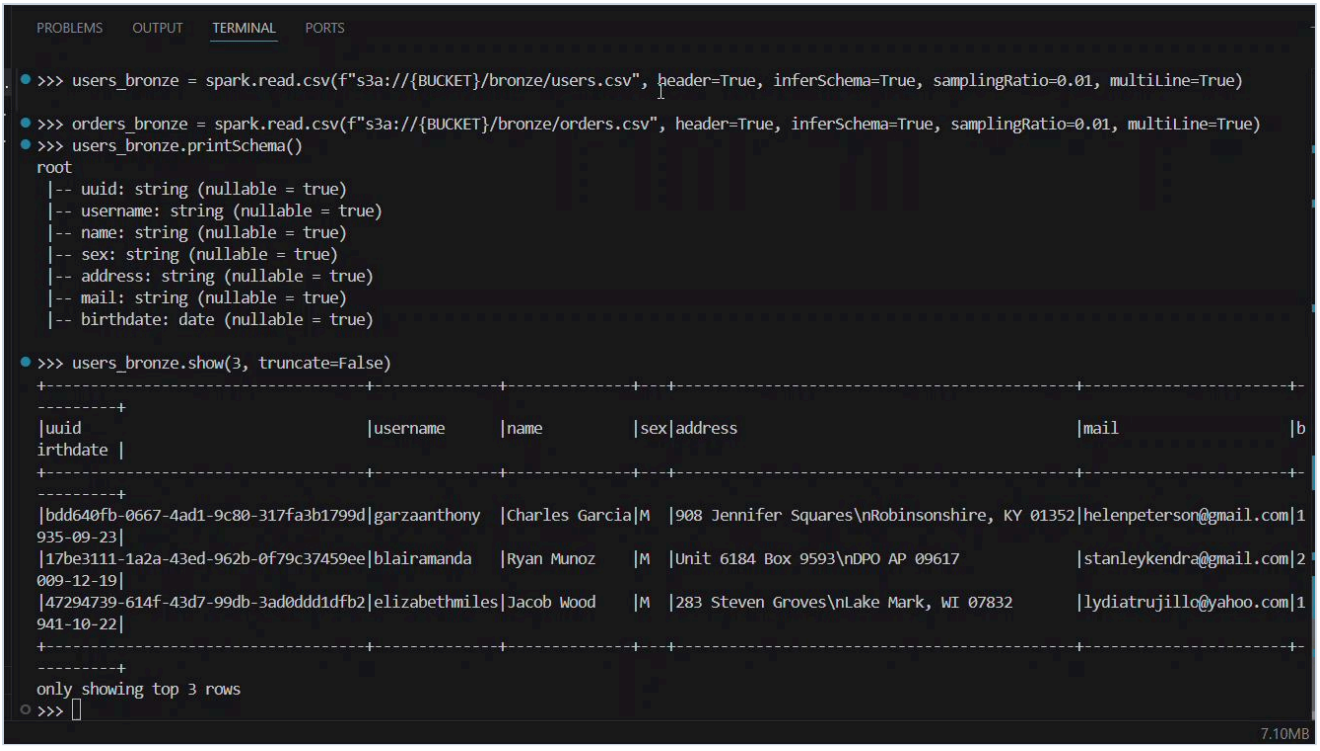


Figure 6 – Schéma inféré (birthdate en date) et premières lignes : l'adresse est lue en un seul champ, avec le retour à la ligne entre la rue et la ville

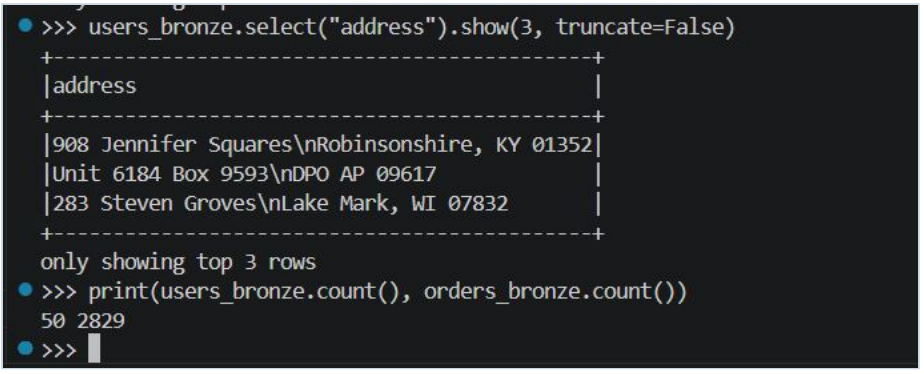


Figure 7 – Colonne address et comptage : 50 utilisateurs et 2829 commandes

L'option `multiLine=True` est indispensable : l'adresse s'étend sur deux lignes physiques, entre guillemets. Sans elle, chaque adresse serait coupée en deux enregistrements. Le décalage des dates de naissance par rapport au sujet (`1935-09-23` au lieu de `1935-09-08`) vient de la date de génération des données, comme au lab DuckDB.

5. Couche silver avec l'API DataFrame

5.1 `stg_users`

```
users_typed = users_bronze.select(
        F.col("uuid").cast("string").alias("user_id"), F.trim("username").alias("username"),
        F.trim("name").alias("name"), F.upper(F.trim("sex")).alias("sex"),
        F.lower(F.trim("mail")).alias("email"), F.col("birthdate").cast("date").alias("birthdat
e"),
        F.split(F.col("address"), "\n").alias("address_lines"))
    .withColumn("street", F.element_at("address_lines", 1))
    .withColumn("address_line_2", F.element_at("address_lines", 2))
stg_users = ... F.regexp_extract("address_line_2", "^(.+), [A-Z]{2} [0-9]{5}$", 1).alias("city")
...
stg_users = stg_users.withColumn("city", F.when(F.col("city") == "", None).otherwise(F.col("cit
y")))
```

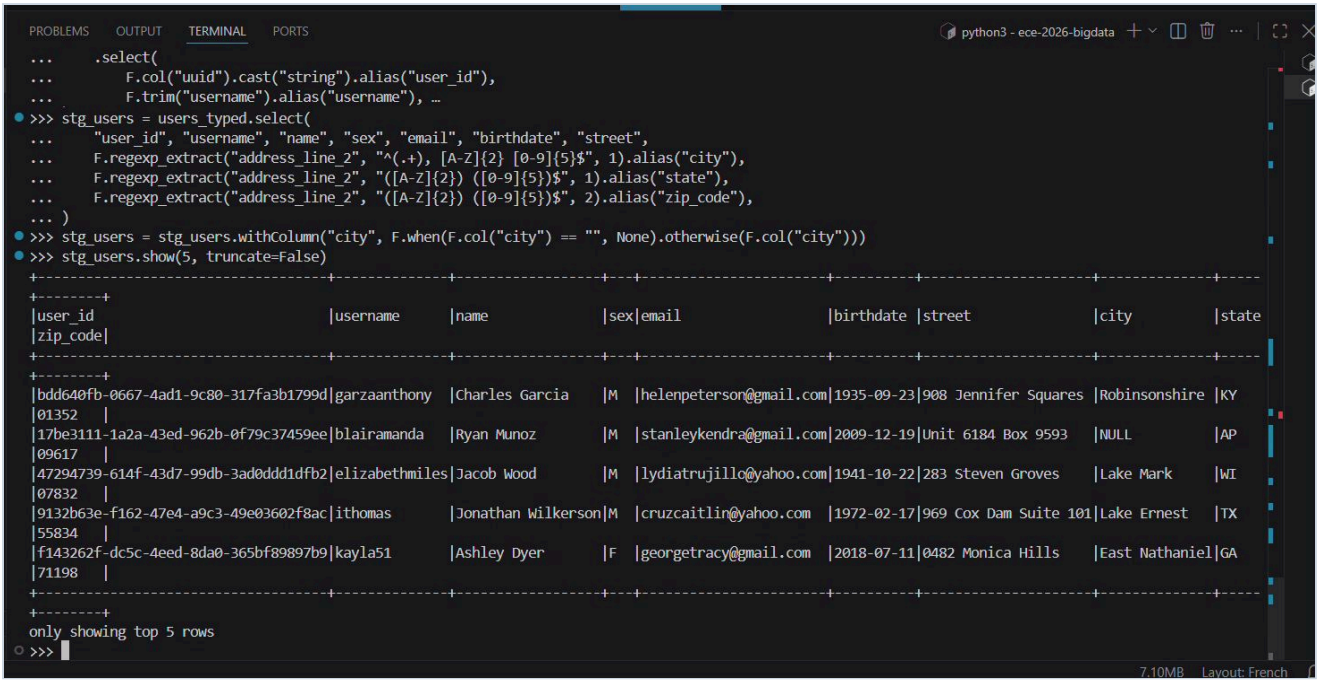


Figure 8 – stg_users : adresse découpée ; pour l'adresse militaire de blairamanda, city vaut NULL

Contrairement au `nullif` du modèle dbt, `regexp_extract` renvoie une **chaîne vide** quand le motif ne correspond pas. Elle est convertie explicitement en `NULL`, faute de quoi elle compterait comme une ville dans un `groupBy`.

5.2 `stg_orders` et dédoublonnage

Trois doublons sont créés volontairement (`orders_typed.limit(3)`), puis supprimés avec une fonction de fenêtre : `row_number()` sur `Window.partitionBy("order_id").orderBy(ordered_at desc)`, dont on ne garde que la ligne n° 1. C'est l'équivalent du `qualify row_number() … = 1` du modèle dbt.

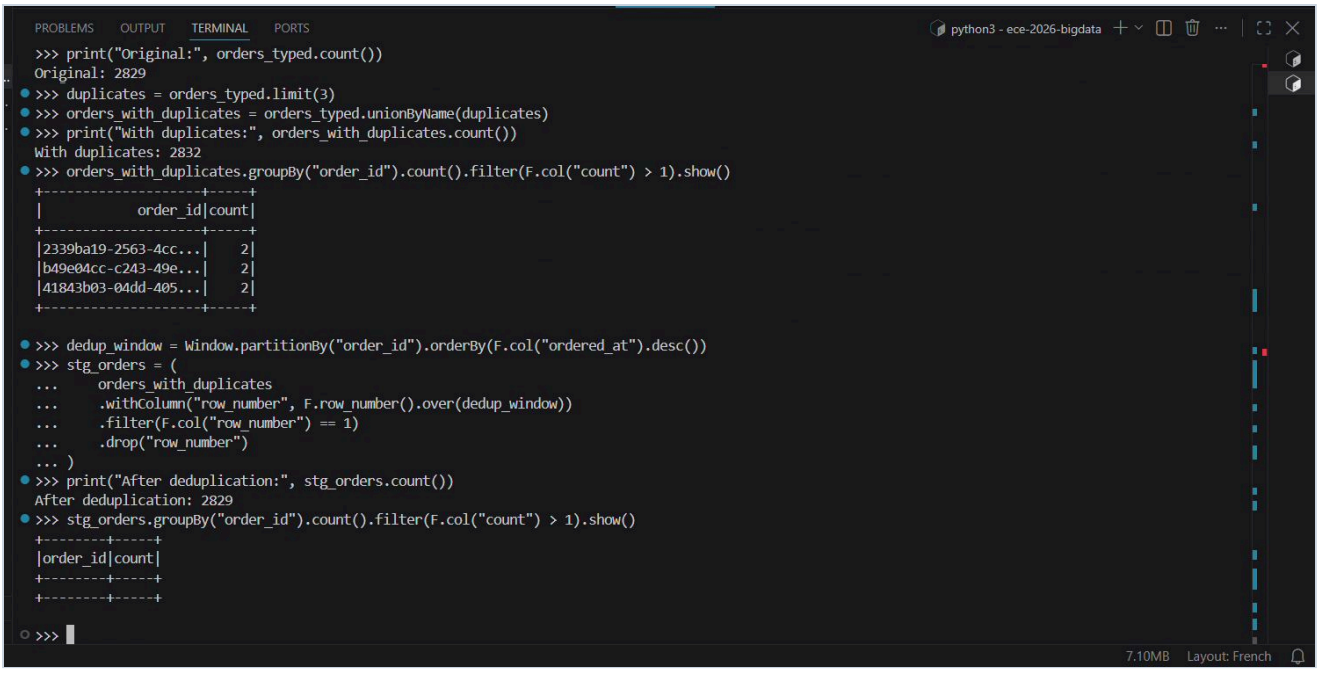


Figure 9 – 2829 commandes, 2832 avec les 3 doublons, puis 2829 après dédoublonnage ; plus aucun order_id en double

6. La même transformation en Spark SQL

```
orders_typed.createOrReplaceTempView("orders_typed")
stg_orders_sql = spark.sql(
    "SELECT order_id, user_id, ordered_at, quantity, product "
    "FROM (SELECT *, row_number() OVER (PARTITION BY order_id ORDER BY ordered_at DESC) AS rn "
    "      FROM orders_typed) WHERE rn = 1")
```

```
>>> orders_typed.createOrReplaceTempView("orders_typed")
>>> stg_orders_sql = spark.sql("""
...     SELECT order_id, user_id, ordered_at, quantity, product
...     FROM (
...         SELECT *, row_number() OVER (PARTITION BY order_id ORDER BY ordered_at DESC) AS rn
...         FROM orders_typed
...     )
...     WHERE rn = 1
... """)
>>> stg_orders_sql.show(5)
+--------------------+--------------------+-------------------+--------+---------+
|            order_id|             user_id|         ordered_at|quantity|  product|
+--------------------+--------------------+-------------------+--------+---------+
|00060d5a-02b3-494...|f53d4330-cdda-44b...|2020-01-28 12:31:...|       2|   cookie|
|0011be30-e268-492...|5f733a3e-5d28-4aa...|2020-04-18 22:51:...|       5|    bread|
|002394e2-2ad9-468...|90164161-cfa7-41c...|2020-02-24 01:03:...|       4|   cookie|
|0041f5d4-9549-463...|aab612c9-415d-474...|2020-03-17 18:29:...|       2|croissant|
|00684d9c-d2bf-4c3...|97fa7f04-8363-400...|2020-03-30 13:36:...|       4|    bread|
+--------------------+--------------------+-------------------+--------+---------+
only showing top 5 rows
>>>
```

Figure 10 – Dédoublonnage écrit en Spark SQL

```
>>> assert stg_orders.count() == stg_orders_sql.count()
>>> assert set(stg_orders.columns) == set(stg_orders_sql.columns)
>>> print("DataFrame et SQL identiques :", stg_orders.count(), stg_orders_sql.count())
DataFrame et SQL identiques : 2829 2829
>>> stg_users.createOrReplaceTempView("stg_users")
>>> spark.sql("""
...     SELECT state, count(*) AS users, sum(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS missing_city
...     FROM stg_users
...     GROUP BY state
...     ORDER BY users DESC
... """).show(10)
+-----+-----+------------+
|state|users|missing_city|
+-----+-----+------------+
|   AE|    3|           3|
|   FM|    3|           0|
|   NJ|    2|           0|
|   VT|    2|           0|
|   DE|    2|           0|
|   VI|    2|           0|
|   NM|    2|           0|
|   PA|    2|           0|
|   TX|    2|           0|
|   AP|    2|           2|
+-----+-----+------------+
only showing top 10 rows
>>>
```

Figure 11 – Les assert passent (2829 = 2829) ; utilisateurs et villes manquantes par État

Réponses aux questions

Q3. Expliquez le résultat du code qui vient d'être exécuté.

La requête compte, pour chaque État, le nombre d'utilisateurs et le nombre d'utilisateurs **sans ville**, puis trie par nombre décroissant. Les 50 utilisateurs sont répartis presque uniformément (2

ou 3 par État), car le générateur tire l'État au hasard. Seuls `AE` **(3 sur 3)** et `AP` **(2 sur 2)** ont des villes manquantes : ce sont les codes **militaires** (*Armed Forces Europe* et *Pacific*), dont les adresses (`DPO AP 09617`) n'ont pas de ville. La conversion de la chaîne vide en `NULL` fonctionne donc, et ne touche que ces adresses. Avant d'exécuter ce SQL, les deux `assert` ont confirmé que les versions DataFrame et SQL donnent le même nombre de lignes et les mêmes colonnes.

Q4. DataFrame ou SQL : laquelle pour une exploration ponctuelle, laquelle pour un modèle réutilisé par plusieurs collègues ?

Les deux sont compilées en **le même plan Catalyst** : les performances sont identiques. Pour une **exploration ponctuelle**, **SQL** est plus court, plus lisible et connu de tous. Pour un **modèle partagé et maintenu**, l'**API DataFrame** est préférable : la logique se découpe en fonctions Python **réutilisables et testables unitairement**, les erreurs de colonnes apparaissent plus tôt, la factorisation est facile et l'éditeur propose l'autocomplétion. Une longue requête SQL dans une chaîne Python est plus difficile à tester et à faire évoluer.

Q5. regexp_extract() renvoie une chaîne vide au lieu de NULL. Où ailleurs ce comportement pourrait-il produire silencieusement un résultat faux ?

Dans `state` **et** `zip_code`, qui utilisent aussi `regexp_extract` mais n'ont pas été convertis. Pour une adresse ne correspondant pas au motif, elles vaudraient `""` : un contrôle `isNull()` ou `not_null` ne la détecterait pas, un `groupBy("state")` créerait un faux État vide, et une jointure avec un référentiel d'États échouerait sans erreur visible. Ici, toutes les adresses correspondent au motif, mais rien ne le garantit pour de nouvelles données : il faudrait appliquer la même conversion ou tester `col != ""`.

7. Écriture de la couche silver en Parquet

```
>>> stg_users.write.mode("overwrite").parquet(f"s3a://{BUCKET}/silver/spark/users.parquet")
[Stage 82:>                                                         (0 + 1) / 1]
>>> stg_orders.write.mode("overwrite").parquet(f"s3a://{BUCKET}/silver/spark/orders.parquet")
>>> print("Écriture terminée")
Écriture terminée
>>> cd ~/work/ece-2026-bigdata
  File "<stdin>", line 1
    cd ~/work/ece-2026-bigdata
       ^
SyntaxError: invalid syntax
>>> export LAB_BUCKET_NAME="$KUBERNETES_NAMESPACE"
  File "<stdin>", line 1
    export LAB_BUCKET_NAME="$KUBERNETES_NAMESPACE"
           ^^^^^^^^^^^^^^^
SyntaxError: invalid syntax
>>> aws s3 --profile default ls "s3://$LAB_BUCKET_NAME/silver/" --recursive
  File "<stdin>", line 1
    aws s3 --profile default ls "s3://$LAB_BUCKET_NAME/silver/" --recursive
        ^^
SyntaxError: invalid syntax
>>>
```

Figure 12 – Écriture de stg_users et stg_orders en Parquet ; les commandes bash collées dans Python sont refusées (difficulté ②)

```
>>> import subprocess
>>> def sh(cmd):
...     print(subprocess.run(cmd, shell=True, capture_output=True, text=True, env={**os.environ, "LAB_BUCKET_NAME": BUCKET}).stdout)
... sh('aws s3 --profile default ls "s3://$LAB_BUCKET_NAME/silver/" --recursive')
  File "<stdin>", line 3
    sh('aws s3 --profile default ls "s3://$LAB_BUCKET_NAME/silver/" --recursive')
    ^^
SyntaxError: invalid syntax
>>> import subprocess; sh = lambda cmd: print(subprocess.run(cmd, shell=True, capture_output=True, text=True, env={**os.environ, "LAB_BUCKET_NAME": BUCKET}).stdout)
>>> sh('aws s3 --profile default ls "s3://$LAB_BUCKET_NAME/silver/" --recursive')
2026-11-05 12:33:04      72929 silver/dbt/orders.parquet
2026-11-05 12:32:36       7018 silver/dbt/users.parquet
2026-11-05 17:30:57          0 silver/spark/orders.parquet/
2026-11-05 17:30:58      11798 silver/spark/orders.parquet/_SUCCESS
2026-11-05 17:30:58     139556 silver/spark/orders.parquet/part-00000-1ec91cbe-43e6-400a-934a-acb0d0cea587-c000.snappy.parquet
2026-11-05 17:30:54          0 silver/spark/users.parquet/
2026-11-05 17:30:57      11783 silver/spark/users.parquet/_SUCCESS
2026-11-05 17:30:56       9581 silver/spark/users.parquet/part-00000-1a48f617-d70e-49bd-a317-26b16e2d99fc-c000.snappy.parquet

>>>
```

Figure 13 – silver/dbt/ : un fichier unique par modèle ; silver/spark/ : un dossier par modèle, avec _SUCCESS et un part-file

```
>>> import subprocess; sh = lambda cmd: print(subprocess.run(cmd, shell=True, capture_output=True, text=True, env={**os.environ, "LAB_BUCKET_NAME": BUCKET}).stdout)
>>> sh('aws s3 --profile default ls "s3://$LAB_BUCKET_NAME/silver/" --recursive')
2026-11-05 12:33:04      72929 silver/dbt/orders.parquet
2026-11-05 12:32:36       7018 silver/dbt/users.parquet
2026-11-05 17:30:57          0 silver/spark/orders.parquet/
2026-11-05 17:30:58      11798 silver/spark/orders.parquet/_SUCCESS
2026-11-05 17:30:58     139556 silver/spark/orders.parquet/part-00000-1ec91cbe-43e6-400a-934a-acb0d0cea587-c000.snappy.parquet
2026-11-05 17:30:54          0 silver/spark/users.parquet/
2026-11-05 17:30:57      11783 silver/spark/users.parquet/_SUCCESS
2026-11-05 17:30:56       9581 silver/spark/users.parquet/part-00000-1a48f617-d70e-49bd-a317-26b16e2d99fc-c000.snappy.parquet

>>> users_silver = spark.read.parquet(f"s3a://{BUCKET}/silver/spark/users.parquet")
>>> orders_silver = spark.read.parquet(f"s3a://{BUCKET}/silver/spark/orders.parquet")
>>> assert users_silver.count() == users_bronze.count()
>>> assert orders_silver.count() == stg_orders.count()
>>> print("Parquet OK :", users_silver.count(), orders_silver.count())
Parquet OK : 50 2829
>>>
```

Figure 14 – Relecture des fichiers Parquet : 50 utilisateurs et 2829 commandes, aucune perte

Réponses aux questions

Q6. Combien de part-files sous silver/spark/.../, et pourquoi ?

Un seul par dossier, car Spark écrit un fichier par partition du DataFrame final. Pour `users`, le CSV est petit et `multiLine=True` le rend **non découpable** : il est lu en une seule partition. Pour `orders`, la fonction de fenêtre provoque un shuffle en 200 partitions, mais l'**Adaptive Query Execution** les fusionne en une seule, vu le faible volume. Le fichier `_SUCCESS`, écrit en dernier, signale une écriture complète ; avec le *committer* S3A, il contient aussi un résumé JSON (11 Ko).

Q7. coalesce(1) avant .write produirait un seul fichier. Que perd-on sur un plus gros jeu de données ?

Le **parallélisme** : toutes les données passent par une seule tâche, sur un seul cœur. L'écriture devient lente, la tâche peut manquer de mémoire, et `coalesce` peut même réduire le parallélisme des étapes en amont. Le fichier obtenu, énorme, ne sera ensuite lu que par un seul lecteur. Ce n'est raisonnable que pour un petit résultat destiné à un humain ; sinon, on vise des fichiers de 100 Mo à 1 Go.

8. Écriture de la couche silver en tables Iceberg

```
spark.sql("CREATE NAMESPACE IF NOT EXISTS lab.silver")
stg_users.writeTo("lab.silver.users").createOrReplace()
stg_orders.writeTo("lab.silver.orders").createOrReplace()
```

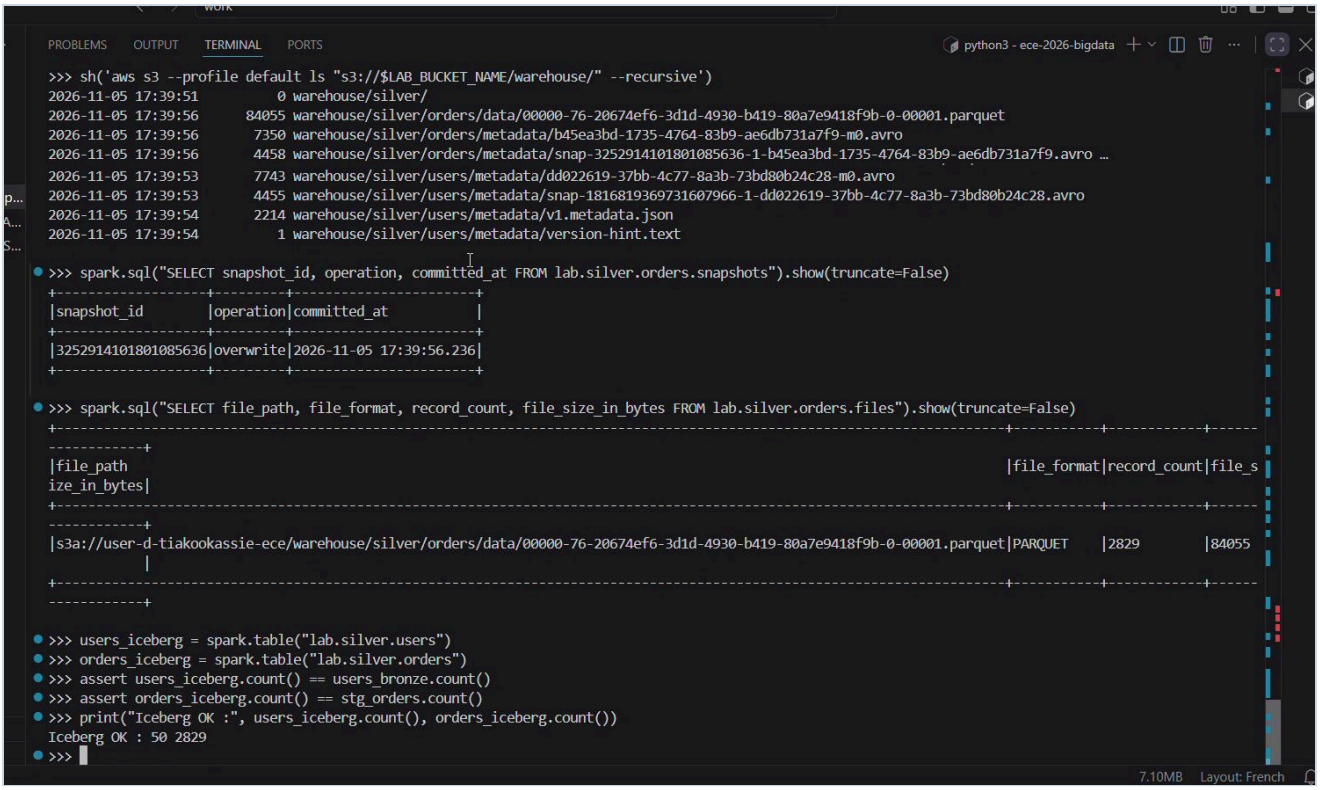


Figure 15 – Fichiers de la table (data/ et metadata/), snapshot initial (overwrite), fichier de données (2829 lignes, 84 Ko) et vérification : 50 et 2829

Le dossier `metadata/` contient `v1.metadata.json` (schéma, partitionnement, liste des snapshots), un **manifeste** `…-m0.avro` (liste des fichiers de données et leurs statistiques), une **manifest list** `snap-….avro` et `version-hint.text` (version courante). Le fichier de données Iceberg (84 Ko) est plus léger que le Parquet de Spark (136 Ko), car Iceberg compresse par défaut en **zstd** au lieu de snappy.

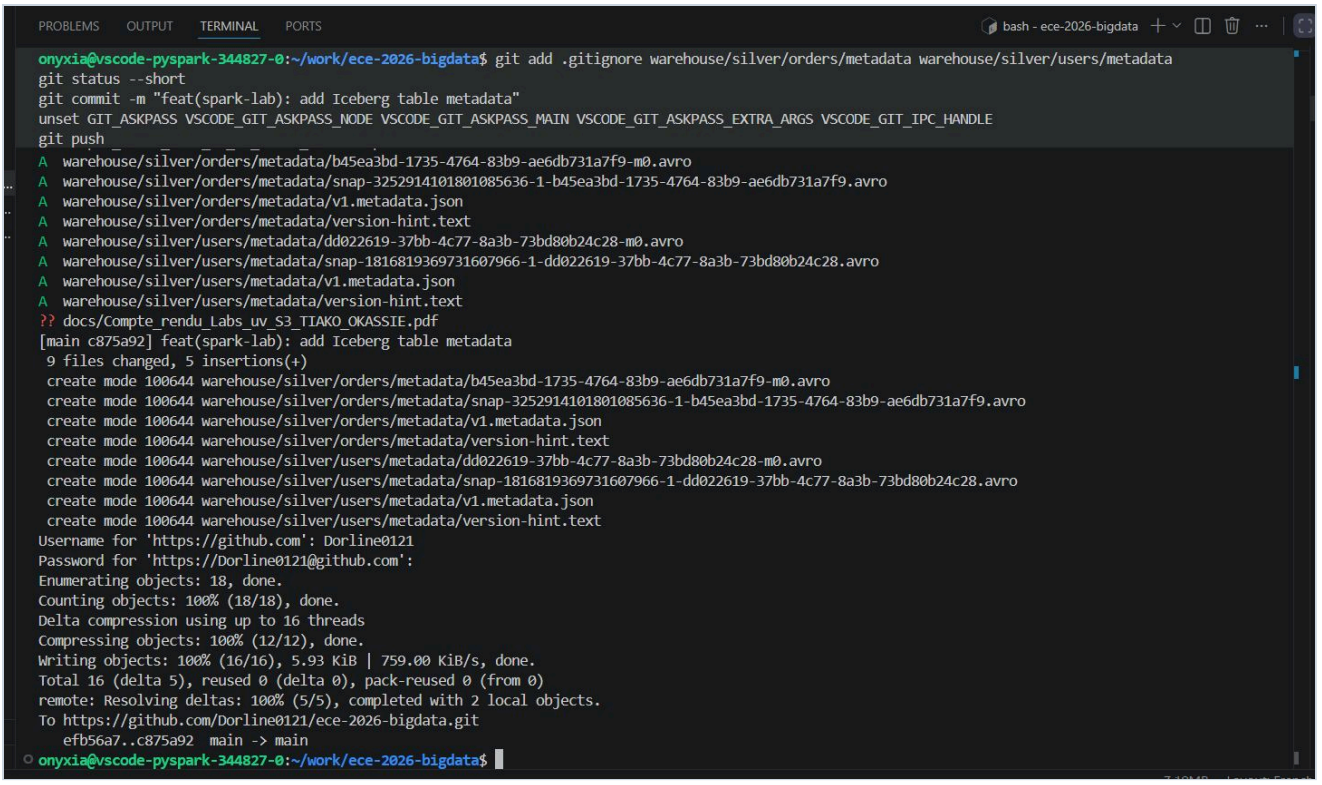


Figure 16 – Commit c875a92 : seules les métadonnées des tables sont versionnées, les fichiers de données sont exclus par .gitignore

Réponses aux questions

Q8. Comparez silver/spark/orders.parquet/ et warehouse/silver/orders/. Quels fichiers contiennent les données, lesquels décrivent la table ?

Dans les deux cas, les **données** sont dans des fichiers **Parquet**. Le dossier Parquet n'a **aucune description** de la table : un lecteur doit lister le dossier et ouvrir les fichiers pour la découvrir. La table Iceberg ajoute un dossier `metadata/` qui la décrit entièrement : schéma et historique, snapshots, liste exacte des fichiers avec leurs statistiques min/max, et version courante. Un lecteur sait donc quels fichiers lire sans lister le stockage objet.

Q9. Un second job écrase le dossier Parquet pendant qu'une requête le lit. Que voit le lecteur ? Qu'est-ce qui change avec Iceberg ?

Avec un dossier Parquet, `overwrite` supprime puis réécrit les fichiers : le lecteur peut voir un dossier vide, un mélange d'anciens et de nouveaux fichiers, ou une `FileNotFoundException`. Avec Iceberg, l'écriture crée de nouveaux fichiers et un nouveau fichier de métadonnées, puis publie la version par un **commit atomique**. Le lecteur travaille sur le snapshot lu au début de sa requête, et voit soit l'ancienne version complète, soit la nouvelle, jamais un état intermédiaire : c'est l'**isolation** des propriétés ACID.

Q10. Avec ce catalogue, le pointeur de version est un fichier du bucket. Que peut-il se passer si deux jobs font un commit en même temps, et qu'apporterait un service de catalogue ?

Le catalogue `hadoop` suppose un renommage atomique que le stockage objet ne garantit pas. Deux jobs simultanés peuvent écrire la même version `vN+1` : le second **écrase** le premier, dont le commit est **perdu silencieusement**. Un **service de catalogue** (REST Catalog, Nessie, Polaris, Glue, Hive Metastore) garde le pointeur dans une base transactionnelle et fait le commit par

compare-and-swap : le second job est rejeté, relit la nouvelle version et réessaie. Il apporte aussi le contrôle d'accès, une liste centrale des tables et le partage entre moteurs.

9. Comparaison avec le lab dbt

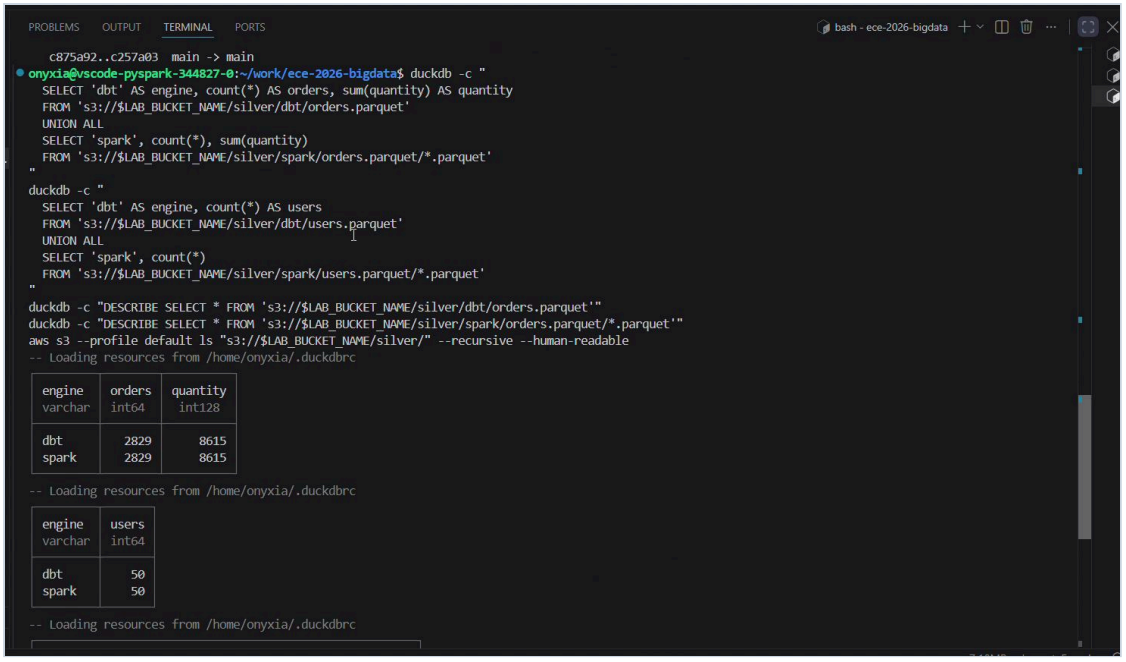


Figure 17 – Mêmes résultats avec les deux moteurs : 2829 commandes, 8615 articles et 50 utilisateurs

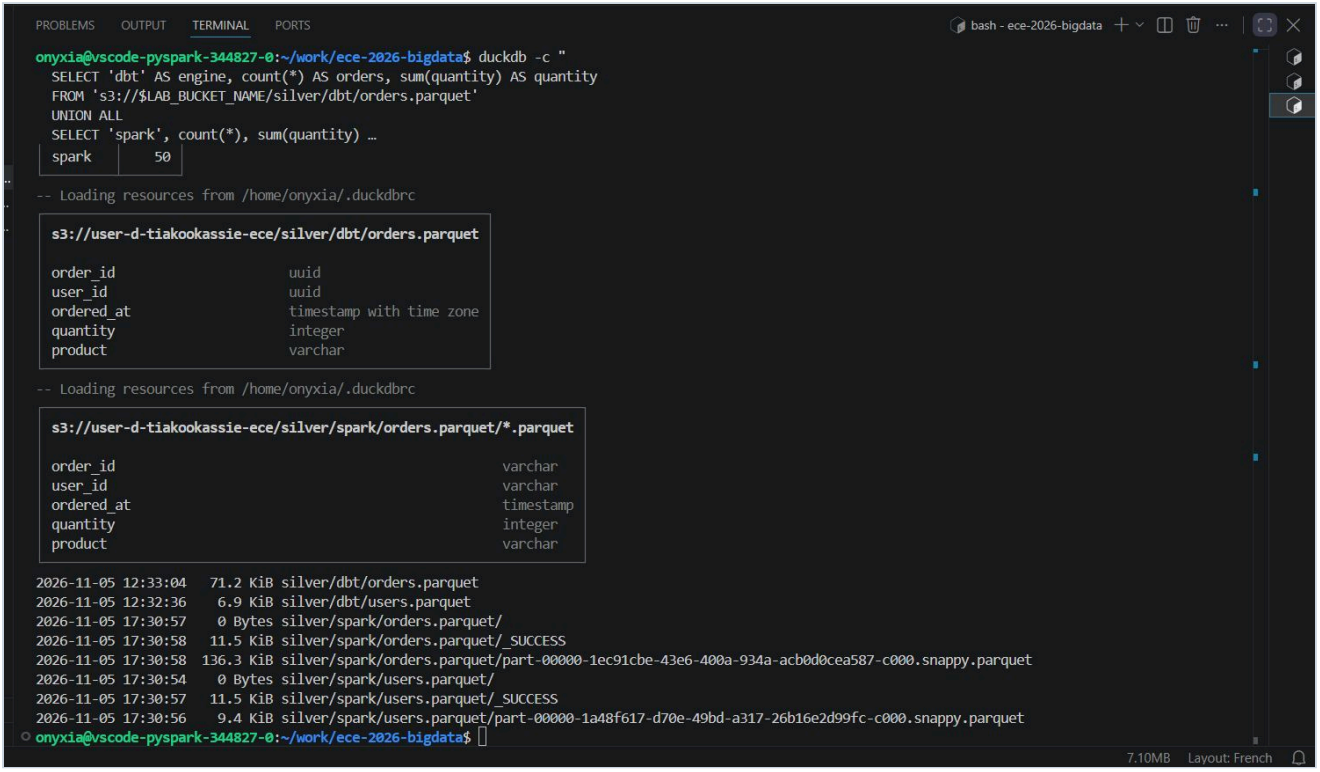


Figure 18 – Schémas et tailles : UUID chez dbt, VARCHAR chez Spark ; fichier unique contre dossier de part-files

	dbt (DuckDB)	Spark
Commandes / articles / utilisateurs	2829 / 8615 / 50	2829 / 8615 / 50
`order_id`, `user_id`	`uuid`	`varchar`
`ordered_at`	`timestamp with time zone`	`timestamp`
Fichier `orders`	71,2 Kio, un seul objet	136,3 Kio, un dossier (part-file et `_SUCCESS`)

Réponses aux questions

Q11. Qu'est-ce qui diffère entre les schémas ? Spark n'a pas de type UUID : qu'avez-vous fait, et quelle conséquence pour les consommateurs ?

Les identifiants sont des `UUID` chez dbt et des `VARCHAR` chez Spark, et la date n'a pas de fuseau chez Spark, qui écrit par défaut ses timestamps au format Parquet **INT96**. Faute de type `UUID`, les identifiants ont été **castés en chaînes**. Conséquences : 36 octets par identifiant au lieu de 16 (le fichier Spark est presque deux fois plus gros), plus aucune validation du format, des jointures plus lentes et sensibles à la casse, et un cast obligatoire pour joindre les deux couches silver. Il faudrait fixer un contrat de schéma commun, et régler `spark.sql.parquet.outputTimestampType` pour les dates.

Q12. Un objet par modèle avec dbt, un dossier de part-files avec Spark : lequel est le plus simple pour un consommateur ? Lequel passe à l'échelle ?

Le **fichier unique** est le plus **simple** : un seul chemin, lisible par n'importe quel outil. Le **dossier de part-files passe à l'échelle** : chaque tâche écrit son fichier en parallèle, la lecture se répartit, et chaque fichier garde une taille raisonnable. En contrepartie, le consommateur doit lire tout le dossier et distinguer une écriture terminée d'une écriture en cours. Un format de table comme **Iceberg** réunit les deux avantages.

Q13. Le lab dbt valide le silver avec des tests. Qu'a utilisé ce lab, et que faudrait-il écrire pour les mêmes garanties ?

Seulement des `assert` sur les nombres de lignes et les colonnes, et un `groupBy().count()` pour les doublons : des vérifications ad hoc, sans rapport ni stockage des lignes fautives. Pour les mêmes garanties, il faudrait une fonction par règle : `unique` (`groupBy(col).count().filter(count > 1)`), `not_null` (`filter(col.isNull())`), `accepted_values` (`filter(~col.isin(...))`) et `relationships` (jointure `left_anti`), regroupées dans des tests `pytest`, ou un framework comme Great Expectations, PyDeequ ou Soda.

Q14. ref() et source() donnent à dbt son graphe de dépendances. Qu'est-ce qui joue ce rôle dans le notebook ?

Les **variables Python** et **l'ordre d'exécution des cellules** (`users_bronze` → `users_typed` → `stg_users`). Spark connaît la lignée de chaque DataFrame par son **plan logique** (visible avec `.explain()`), mais seulement en mémoire, dans la session : il n'y a ni graphe au niveau du projet, ni ordre d'exécution automatique, ni documentation. En production, ce rôle revient à un orchestrateur comme Airflow ou Dagster.

Q15. Comparez le temps de construction avec dbt build et avec les cellules de ce lab. Pourquoi ne sont-ils pas directement comparables ?

`dbt build` prend environ 4 à 5 secondes pour 2 seeds, 2 vues et 19 tests ; Spark demande le démarrage de la JVM, le téléchargement d'Iceberg, puis plusieurs secondes par action. Les chiffres ne sont pas comparables : dbt crée des **vues** sans calcul, alors que Spark **matérialise** les données ; Spark est **paresseux** et recalcule toute la chaîne à chaque action ; sur 340 Ko de données, les **coûts fixes** (JVM, planification, tâches) dominent ; enfin, le temps de saisie entre les cellules s'ajoute.

Q16. dbt avec DuckDB ou Spark, pour : un batch quotidien de 10 Go, un batch quotidien de 10 To, une équipe d'analystes qui ne connaît que SQL ?

- **10 Go par jour** : **dbt avec DuckDB**. Une seule machine traite ce volume rapidement, sans cluster à administrer, en gardant les tests et la documentation de dbt.
- **10 To par jour** : **Spark**. Ce volume dépasse une machine et exige un traitement distribué, avec parallélisme et reprise sur erreur.
- **Des analystes qui ne connaissent que SQL** : **dbt**, dont les modèles sont en SQL avec tests, documentation et lineage intégrés. Si le volume grandit, l'équipe peut changer de moteur grâce aux adaptateurs (dbt-spark, Databricks, Trino) sans réécrire ses modèles.

10. Exercices

Exercice 1 – `birthdate_is_valid` en DataFrame et en SQL

```
first_orders = stg_orders.groupBy("user_id").agg(F.min("ordered_at").alias("first_order_at"))
users_valid_df = (stg_users.join(first_orders, "user_id", "left")
    .withColumn("birthdate_is_valid",
        ~F.coalesce(F.col("first_order_at") < F.col("birthdate").cast("timestamp"), F.lit(False)))
    .drop("first_order_at"))
```

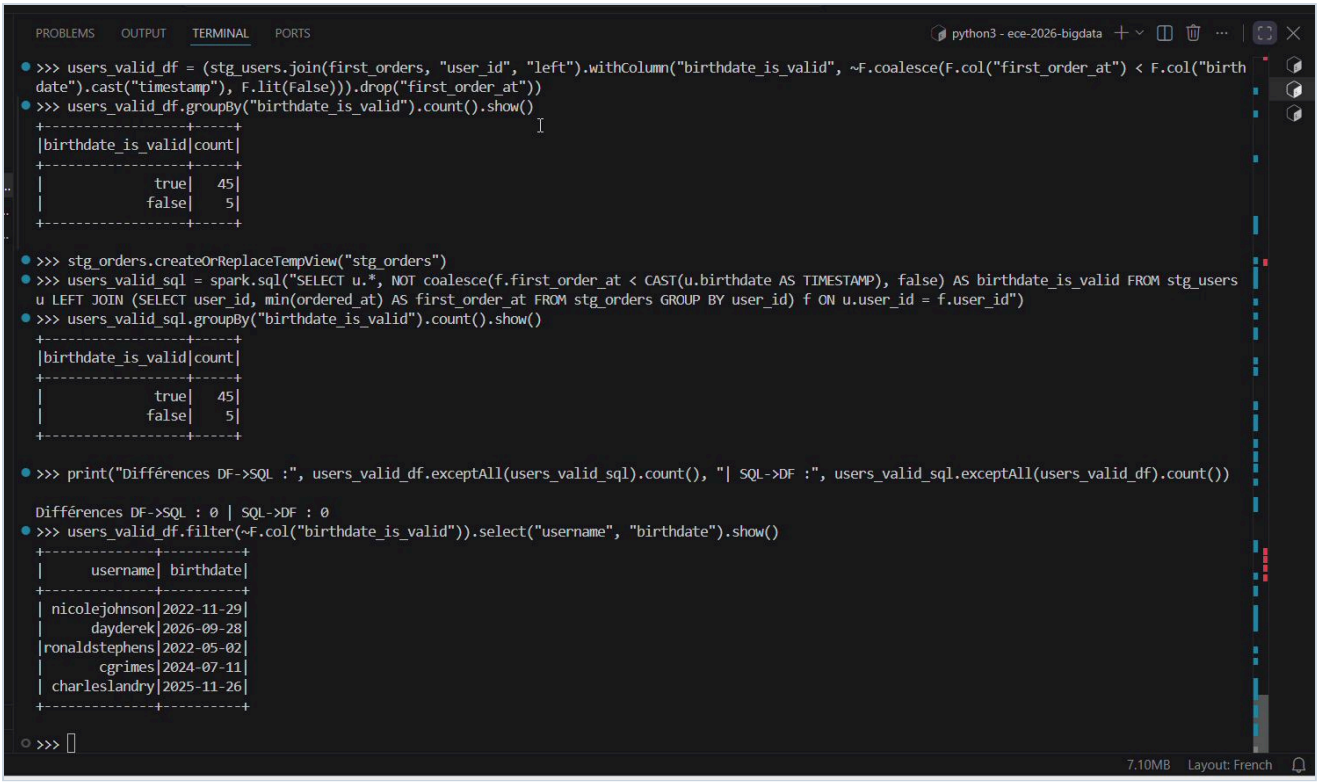


Figure 19 – 45 dates valides et 5 invalides avec les deux versions ; exceptAll vide dans les deux sens ; les 5 utilisateurs concernés

Les deux versions sont **strictement identiques** (`exceptAll` renvoie 0 ligne dans les deux sens). On retrouve les **mêmes 5 utilisateurs** qu'au lab dbt, nés entre 2022 et 2026 alors que leurs commandes datent de 2020. **Comparaison avec dbt** : la logique est identique, mais dans dbt la règle vit dans le modèle `stg_users`, vérifiée par un test à chaque build, et la date invalide y est aussi remplacée par `NULL` ; ici, la règle n'existe que dans une variable de la session.

Exercice 2 – Code postal : UDF Python, `pandas_udf` et `regexp_extract`

```
zip_py = F.udf(lambda a: (lambda m: m.group(1) if m else "")(ZIP_RE.search(a or "")), "string")
zip_pd_fn = lambda s: s.fillna("").str.extract(r"[A-Z]{2} ([0-9]{5})$", expand=False).fillna("")
zip_pd_fn.__annotations__ = {"s": pd.Series, "return": pd.Series}
zip_pd = F.pandas_udf(zip_pd_fn, "string")
zip_native = F.regexp_extract("address", r"[A-Z]{2} ([0-9]{5})$", 1)
```



Figure 20 – Résultats identiques (0 ligne différente), puis temps mesurés sur 50 lignes et sur 1 million de lignes

Version	50 lignes (médiane)	1 million de lignes (meilleur temps)
`regexp_extract` (intégrée)	0,118 s	**1,12 s**
`pandas_udf`	0,146 s	2,15 s (≈ 1,9× plus lente)
UDF Python	0,122 s	3,49 s (≈ 3,1× plus lente)

Sur 50 lignes, les temps sont **indiscernables** : ils sont dominés par les coûts fixes (lecture du CSV sur S3, planification, démarrage du processus Python, d'où le pic à 1,04 s). Sur **1 million de lignes** (les adresses répliquées et mises en cache), le classement est net. La **fonction intégrée** s'exécute dans la JVM, sans sérialisation. L'**UDF Python** envoie chaque ligne, sérialisée, à un processus Python séparé, **une à la fois**. La `pandas_udf` fait le même aller-retour **par lots Arrow**, traités de façon vectorisée par pandas : elle est 1,6 fois plus rapide que l'UDF Python. Règle pratique : d'abord les fonctions intégrées, puis une `pandas_udf`, et l'UDF Python en dernier recours.

Exercice 3 – Mise en cache des données bronze

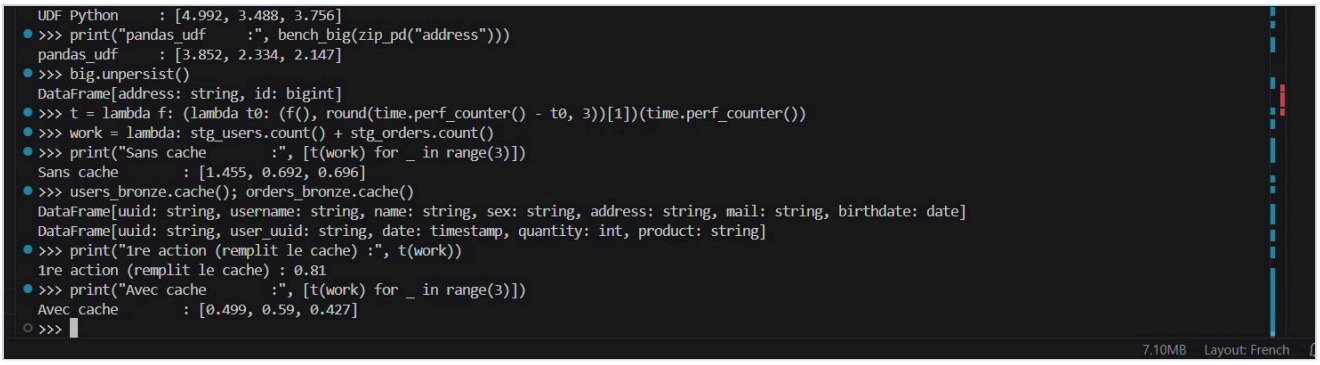


Figure 21 – Temps de stg_users.count() + stg_orders.count() sans cache, à la 1re action, puis avec cache

Mesure	Temps (s)
Sans cache	1,455 · 0,692 · 0,696
1re action, qui remplit le cache	0,81
Avec cache	**0,499 · 0,59 · 0,427**

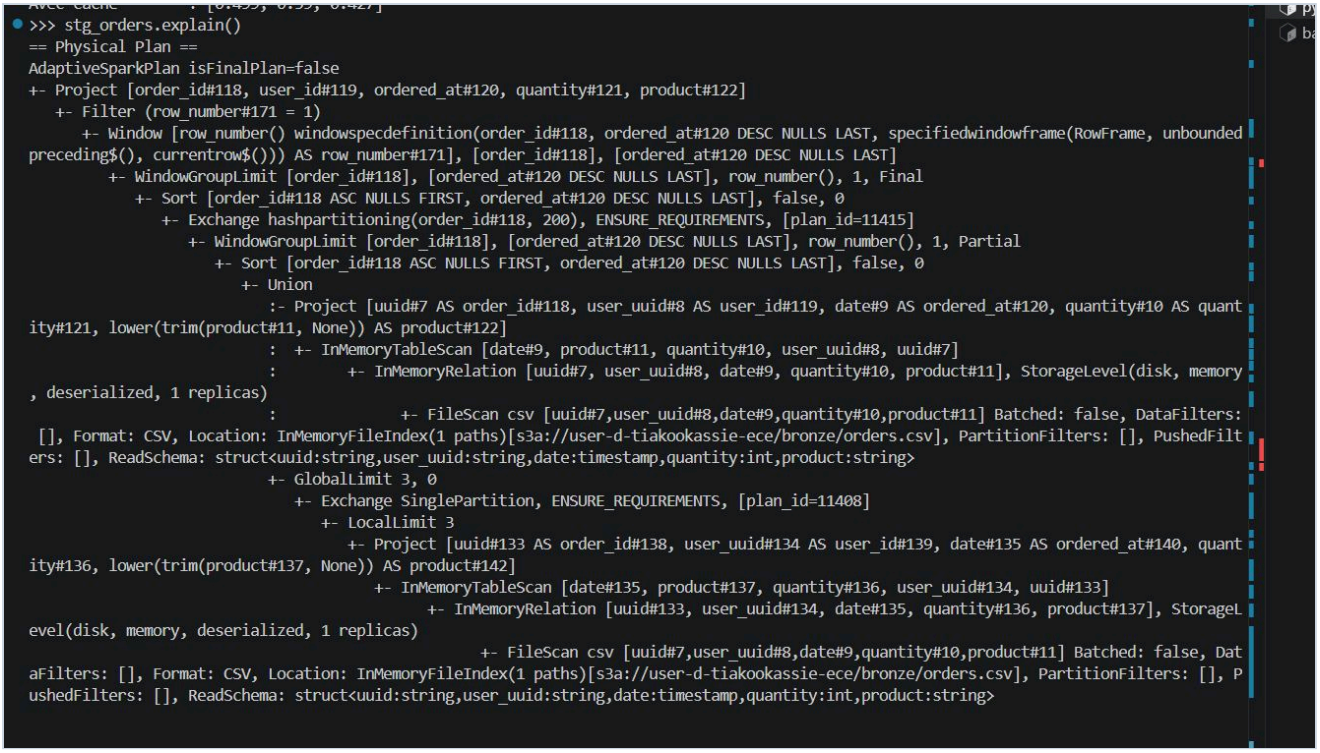


Figure 22 – Plan d'exécution : InMemoryTableScan sur le cache, shuffle de la fonction de fenêtre (Exchange hashpartitioning 200) et optimisation WindowGroupLimit

Avec le cache, les actions sont environ **30 % plus rapides**. La 1re action est un peu plus lente, car elle lit S3 **et** remplit le cache. Le plan confirme que Spark lit désormais les données **en mémoire** (`InMemoryTableScan`, `StorageLevel(disk, memory, deserialized)`) : le `FileScan csv` n'apparaît plus que sous l'`InMemoryRelation`, pour décrire le remplissage initial. Le gain reste limité, car le cache ne supprime que la lecture des CSV : le **shuffle** du dédoublonnage (`Exchange hashpartitioning(order_id, 200)`) est refait à chaque action. Mettre en cache `stg_orders` lui-même éviterait aussi ce calcul. Le plan montre enfin l'optimisation `WindowGroupLimit` (Partial puis Final) : comme seule la ligne n° 1 est gardée, Spark élimine des lignes **avant** le shuffle.

Exercice 4 – Ajout dans Iceberg, *time travel* et comparaison avec Parquet

```
new_order.writeTo("lab.silver.orders").append()
spark.sql(f"SELECT count(*) FROM lab.silver.orders VERSION AS OF {first_snap}")
new_order.write.mode("append").parquet(f"s3a://{BUCKET}/silver/spark/orders.parquet")
```

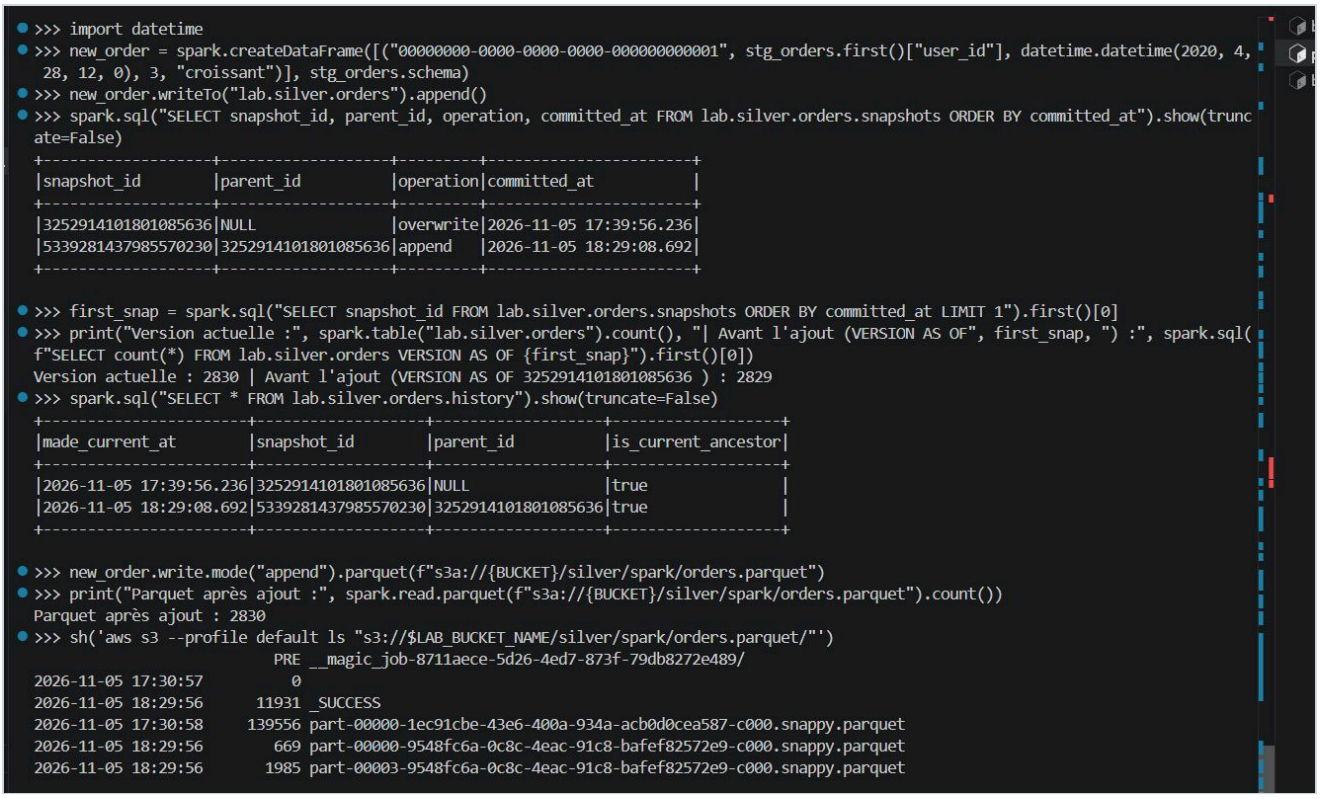


Figure 23 – Iceberg : nouveau snapshot append rattaché au premier, 2830 lignes et 2829 avec VERSION AS OF, historique ; Parquet : 2830 lignes et nouveaux part-files, sans historique

Avec Iceberg, l'ajout crée un **second snapshot** (`append`) dont le `parent_id` est le premier. La table compte 2830 lignes, mais `VERSION AS OF` permet de la relire telle qu'elle était **avant l'ajout**, avec 2829 lignes. La table `history` garde la trace de chaque version : on peut auditer, comparer deux versions ou revenir en arrière (*rollback*).

Avec le dossier Parquet, l'ajout dépose simplement **de nouveaux part-files** : la lecture renvoie 2830 lignes, sans aucun moyen de retrouver l'état précédent. Le listing révèle aussi les limites de cette approche : deux fichiers pour une seule ligne, dont un presque vide (669 octets, une partition vide du petit DataFrame, qui contient seulement le schéma) ; un `_SUCCESS` écrasé, qui ne décrit plus que la dernière écriture ; et un préfixe temporaire `__magic_job-…/` laissé par le *committer* S3A. Pour annuler l'ajout, il faudrait identifier et supprimer les bons fichiers à la main, sans garantie qu'un lecteur concurrent n'ait pas vu un état intermédiaire.

11. Synthèse et conclusion

Étape	Résultat
Session Spark	Spark 4.1.1, S3A avec identifiants temporaires, catalogue Iceberg `lab`
Couche bronze	50 utilisateurs et 2829 commandes lus sur S3
Silver DataFrame et SQL	Résultats identiques ; 3 doublons injectés puis supprimés
Parquet	Un dossier par modèle (1 part-file), relecture sans perte
Iceberg	2 tables, métadonnées versionnées dans Git (commit `c875a92`)
Comparaison avec dbt	Mêmes données ; UUID contre VARCHAR ; fichier Spark presque deux fois plus gros
Exercice 1	45 dates valides, 5 invalides, DataFrame et SQL identiques
Exercice 2	Sur 1 million de lignes : intégrée 1,12 s, `pandas_udf` 2,15 s, UDF Python 3,49 s
Exercice 3	Cache : environ 30 % plus rapide, `InMemoryTableScan` dans le plan
Exercice 4	*Time travel* Iceberg : 2830 lignes et 2829 avec `VERSION AS OF` ; aucun historique en Parquet

Ce lab montre que Spark produit exactement la même couche silver que dbt, avec l'API DataFrame ou avec Spark SQL, qui partagent le même optimiseur. Il demande en revanche plus de configuration (connecteur S3A, identifiants temporaires, paquets) et une rigueur particulière sur les types : absence d'`UUID`, chaîne vide renvoyée par `regexp_extract`. Les exercices ont mis en évidence deux leviers de performance : préférer les fonctions intégrées aux UDF, et mettre en cache les données réutilisées.

La comparaison entre un dossier Parquet et une table Iceberg est l'enseignement principal. Parquet n'est qu'un ensemble de fichiers, sans transaction ni historique ; Iceberg ajoute une couche de métadonnées qui apporte des écritures atomiques, l'isolation des lecteurs et le *time travel*. C'est cette couche qui fait passer d'un data lake à un **lakehouse**. Enfin, sur des volumes aussi petits, dbt avec DuckDB reste plus simple et plus rapide ; Spark prend tout son sens lorsque les données dépassent les capacités d'une seule machine.